# Clustering III: Spectral Clustering and Validation

# From LE to Spectral Clustering

You already know **Laplacian Eigenmaps**: build a neighborhood graph and use its low-frequency eigenvectors as coordinates.

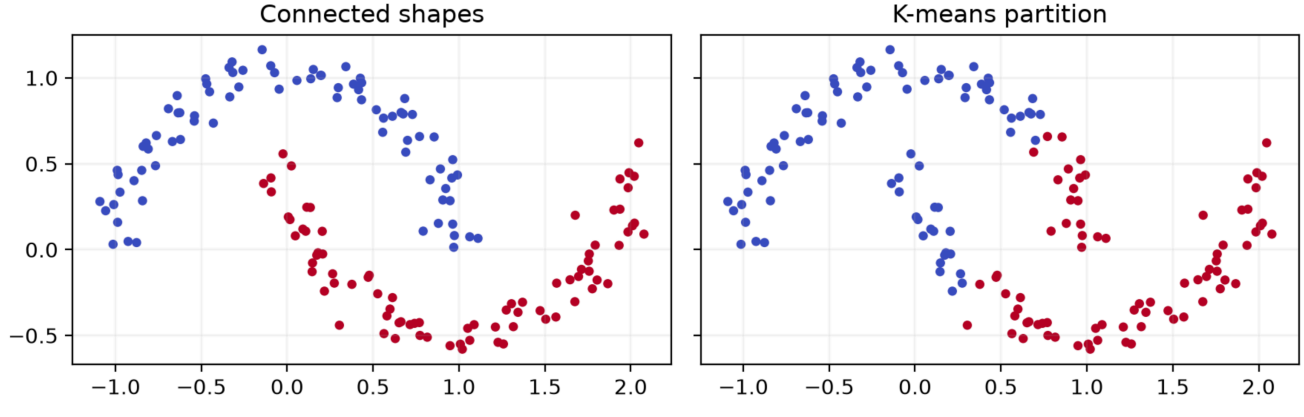
Spectral clustering takes one more step: **cluster those coordinates**.

$$\text{points} \to \text{similarity graph} \to \text{eigenvectors} \to \text{k-means labels}$$

## The problem with centers on curved groups

K-means prefers groups that are compact around a mean. Two interlocking moons are connected locally but have overlapping centers.

**Question:** should a cluster be defined by its centroid or by paths of strong neighbor links?



# Replace distances to centers with a graph

A point is a **vertex**. A strong pairwise similarity is an **edge**. Dense connections within a group and weak connections between groups suggest a partition.

$$W_{ij} \geq 0, \quad W_{ij} = W_{ji}, \quad W_{ii} = 0$$

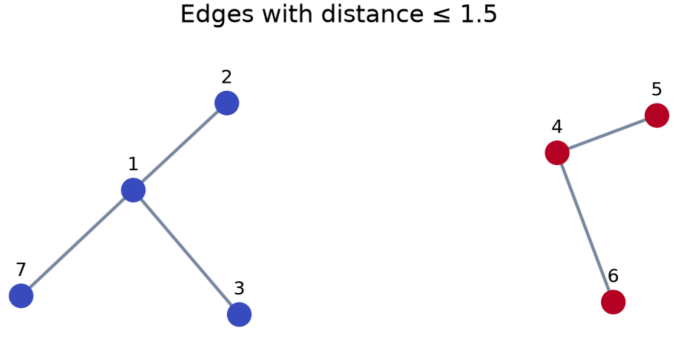
The graph records **local relationships**, so the cluster boundary can curve.

## the edge threshold changes the graph

Raise the distance threshold to add edges. Too small: isolated fragments. Too large: the two groups merge.

**Look for:** a range that preserves within-group links without many cross-group links.

Distance threshold 1.5



# How to assign edge weights

A Gaussian affinity gives nearby points stronger links:

$$W_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

Usually we retain only **k nearest neighbors** (or points within a radius). $\sigma$ and graph sparsity change the result; feature scaling still matters.

# Degree and volume: how much connection?

The **degree** of vertex $i$ is the sum of its edge weights. The volume of a set adds its degrees.

$$d_i = \sum_j W_{ij}, \qquad \mathrm{vol}(A) = \sum_{i \in A} d_i$$

Volume measures how strongly a group participates in the graph. It will stop us rewarding tiny isolated clusters.

# The graph Laplacian

Define $D = \mathrm{diag}(d_1, \dots, d_n)$ and $L = D - W$.

$$f^T L f = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2$$

A low-energy vector changes slowly across **strong** edges. Constant values on a connected component have zero energy.

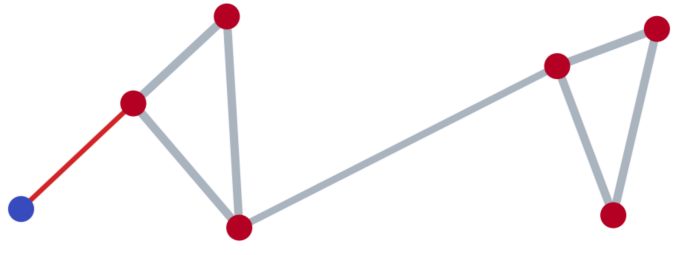
# choose a cut

A **cut** is the total weight of edges crossing a proposed partition. Move between a leaf split and a split of the two dense groups.

$$\mathrm{cut}(A, \bar{A}) = \sum_{i \in A, j \in \bar{A}} W_{ij}$$

0: isolate leaf · 1: split dense groups 0

Cut weight = 0.2

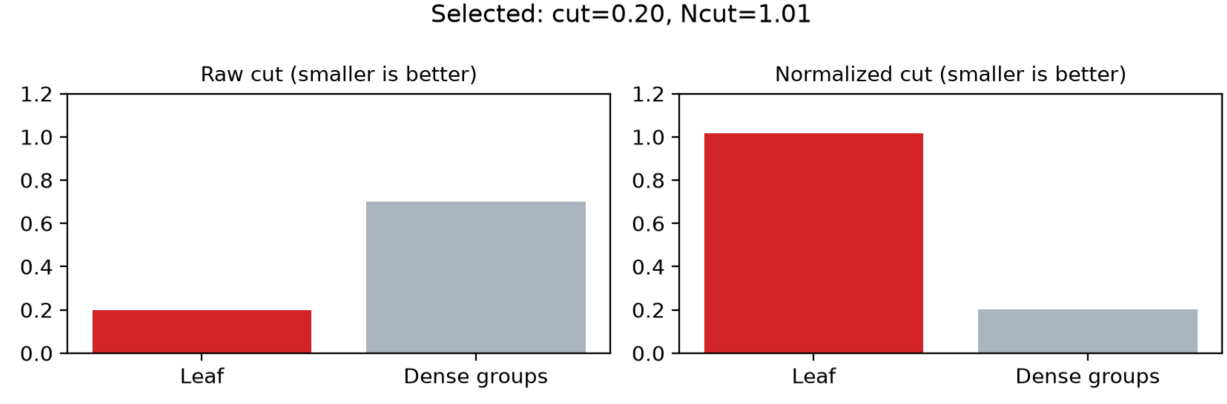


## Cut can prefer a tiny group

The single leaf has the **smaller cut**, but its normalized cut is much worse. The denominator penalizes separating a low-volume fragment.

$$\mathrm{Ncut}(A, \bar{A}) = \frac{\mathrm{cut}(A, \bar{A})}{\mathrm{vol}(A)} + \frac{\mathrm{cut}(A, \bar{A})}{\mathrm{vol}(\bar{A})}$$

Selected partition 0



# From a hard partition to a continuous vector

The discrete Ncut problem is combinatorial. Its relaxation replaces a binary group indicator with a real-valued vector.

For the normalized Laplacian $L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$, low eigenvectors minimize a smoothness objective under normalization.

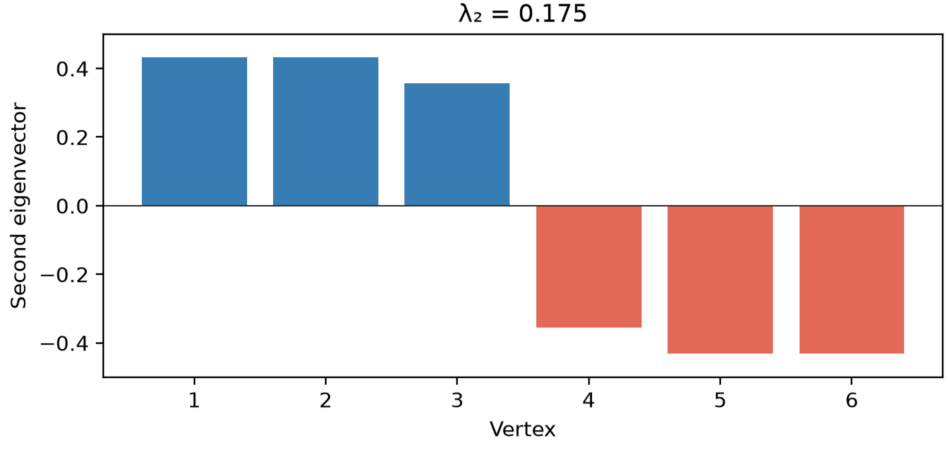
**Then:** discretize the embedding into labels with k-means.

## the Fiedler vector follows the bridge

For this connected graph, the second eigenvector of $L$ separates its two dense halves. Increase bridge weight and the split becomes less pronounced.

**Sign is arbitrary:** swapping positive and negative leaves the partition unchanged.

Bridge weight 0.3



# The link to Laplacian Eigenmaps

Both methods encode **strong neighbors as nearby coordinates**.

- Laplacian Eigenmaps uses the low eigenvectors as a representation.
- Spectral clustering applies a final **partition** in that representation.

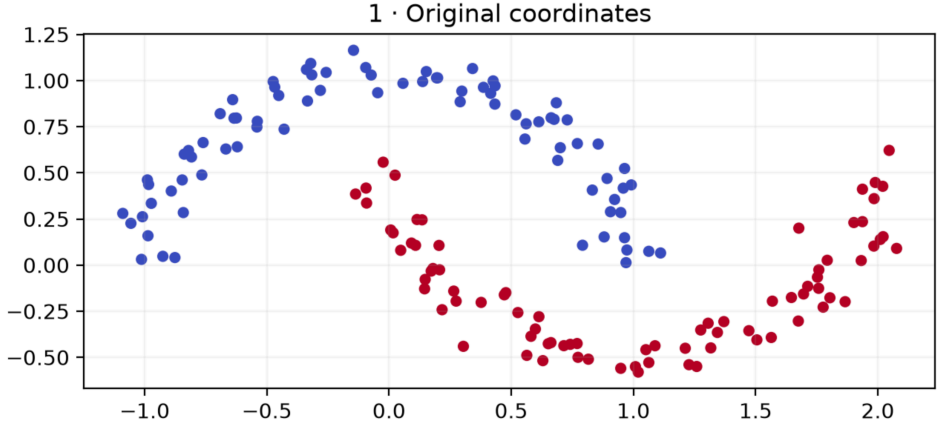
The exact eigenproblem depends on normalization: $L$, $(L, D)$ or $L_{\mathrm{sym}}$. Do not mix coordinates from one formulation with another.

## a spectral embedding unfolds the moons

Step through the raw data, the graph, and its two-dimensional low-frequency embedding. The embedding makes the two groups easier for k-means to separate.

For the symmetric normalized Laplacian, row-normalize the selected eigenvectors before k-means.

Stage 0



# Spectral clustering: the practical recipe

1. Scale features and choose a similarity graph $W$.
2. Form a graph Laplacian; select $k$ low eigenvectors.
3. Form one $k$-dimensional row per point; row-normalize for the $L_{\mathrm{sym}}$ recipe.
4. Run k-means on these rows; return labels for the **original** points.

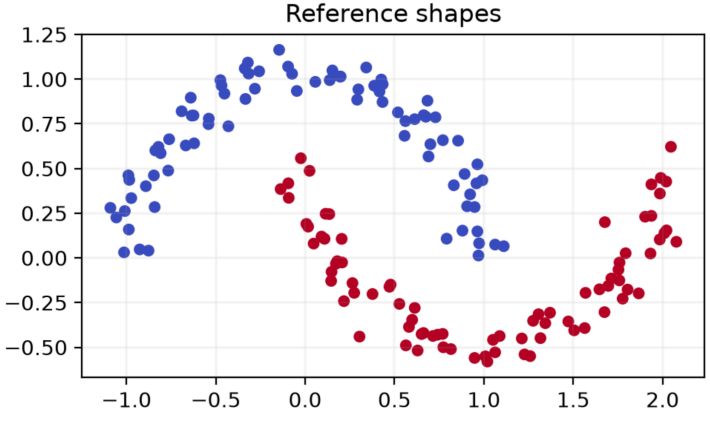
**Key choice:** the graph can matter more than the final k-means run.

# k-means versus spectral on two moons

Move from the true shapes to k-means labels, then spectral labels. Both methods use $k = 2$; they differ in the space where k-means operates.

**Watch:** spectral clustering follows local connectivity.

Partition 0



# Graph design can change the answer

- **Few neighbors:** fragile or disconnected graph.
- **Too many neighbors:** shortcuts across the gap between groups.
- **Gaussian width $\sigma$:** very small weights can effectively delete edges; very large widths flatten similarities.

Inspect graph connectivity and repeat the analysis over plausible settings.

# Limits and computational cost

A dense affinity matrix needs $O(n^2)$ storage, and a full dense eigendecomposition can cost $O(n^3)$ time. Sparse neighbor graphs and partial eigensolvers help at scale.

Disconnected graphs have multiple zero eigenvalues. Eigenvectors can rotate within that zero eigenspace, so interpret the **subspace/partition**, not one vector entry by entry.

# Spectral clustering: three takeaways

**Idea:** a cluster can be a well-connected region of a graph.

**Math:** low Laplacian eigenvectors encode weakly connected groups.

**Practice:** graph construction, normalization, $k$, and final k-means all require choices. Now we ask how to assess them.

# Validation: what are we checking?

**Internal:** compactness and separation from the data alone (inertia, silhouette).

**External:** agreement with trusted reference labels (ARI), if labels exist.

**Stability:** whether small changes to data, seed, or graph choices change the partition.

No single score proves that clusters are meaningful for a task.

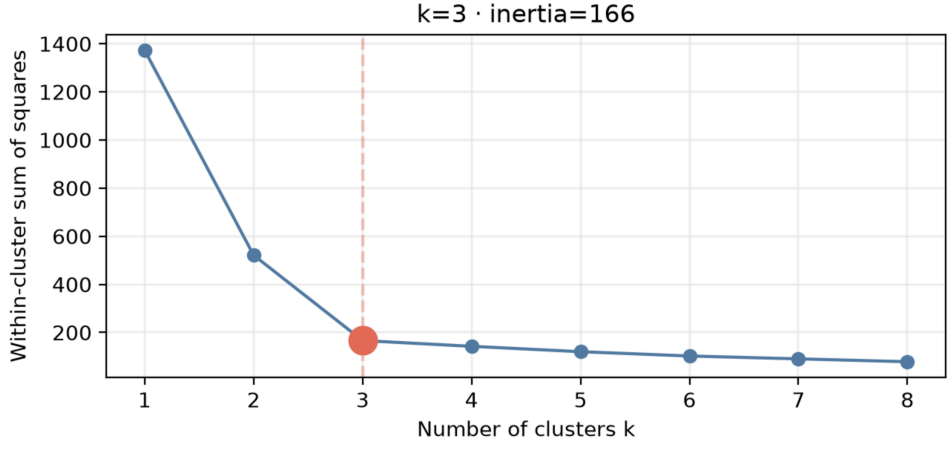
# the elbow is a model-selection clue

Inertia always decreases as $k$ increases. Select $k$ and look for a **bend** where extra groups buy diminishing improvement.

$$J_k = \sum_{r=1}^{k} \sum_{i \in C_r} \|x_i - \mu_r\|^2$$

The elbow is a heuristic; some datasets have no clear bend.

Number of clusters k 3



# Silhouette: a point-level question

For point $i$, let $a_i$ be mean distance to its **own** cluster and $b_i$ the smallest mean distance to **another** cluster.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$$

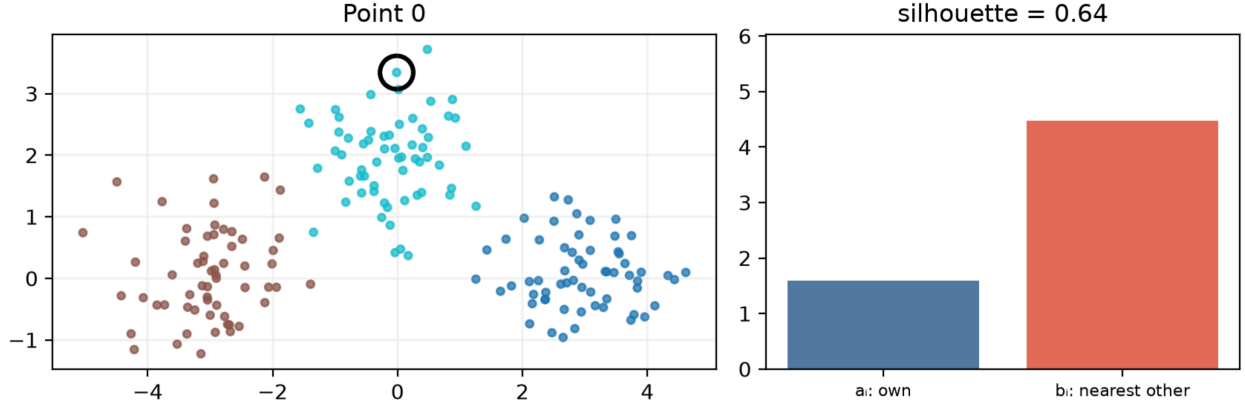
Near $1$: well placed. Near $0$: boundary. Below $0$: another cluster is closer on average. Singleton clusters use $s_i = 0$ by convention.

## inspect one silhouette point

Select a point. The circle marks its own cluster; $a_i$ and $b_i$ compare mean distances, not just the nearest neighbor.

A negative silhouette suggests the assignment deserves inspection.

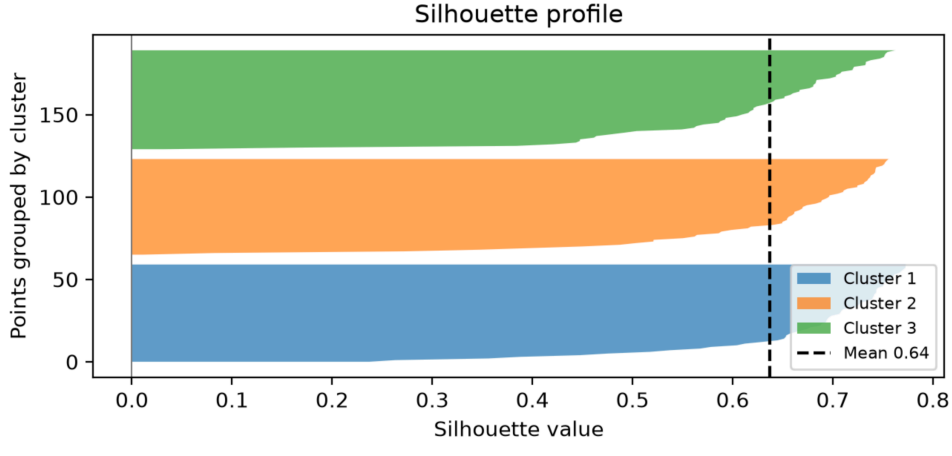
Example point 0



## From one silhouette to a full partition

Plot point silhouettes within each group. Look for a broadly positive distribution and any cluster with many negative members.

The mean silhouette is convenient, but inspect **cluster-level shapes and sizes** too.



# When reference labels exist: ARI

The **Adjusted Rand Index** compares pairwise agreements between two partitions and corrects for chance agreement.

- $1$: identical partitions, even if label names are swapped.
- About $0$: chance-level agreement under the adjustment model.
- It can be **negative** for worse-than-chance agreement.

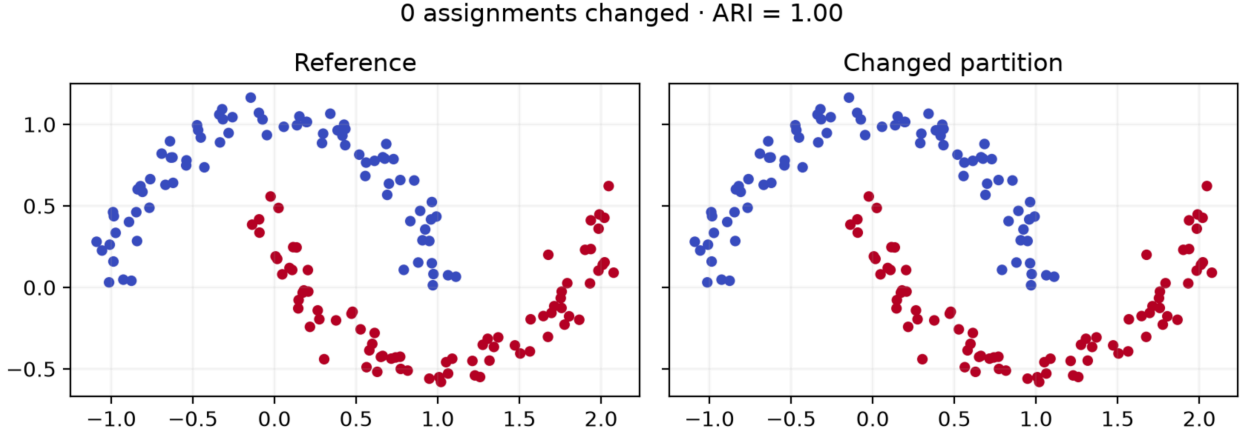
Reference classes are only a useful target if they match the clustering goal.

## ARI falls as assignments change

Start from the two true moon groups, then flip an increasing number of assignments. ARI responds to the **partition**, not the numeric names of labels.

The same score can hide different error patterns; always inspect the points.

Assignments flipped 0

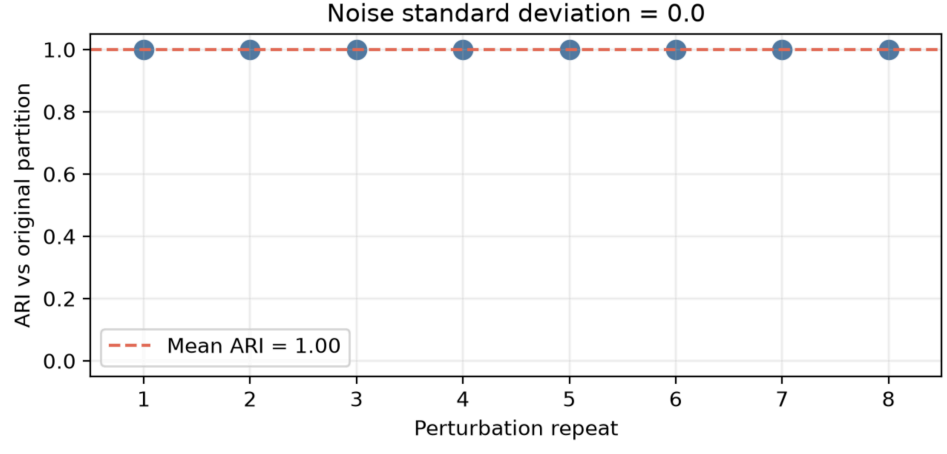


# test partition stability

Add measurement noise, refit k-means eight times, and compare each partition with the original using ARI.

**Stable** means the result survives this perturbation. It does not by itself mean the groups answer the scientific question.

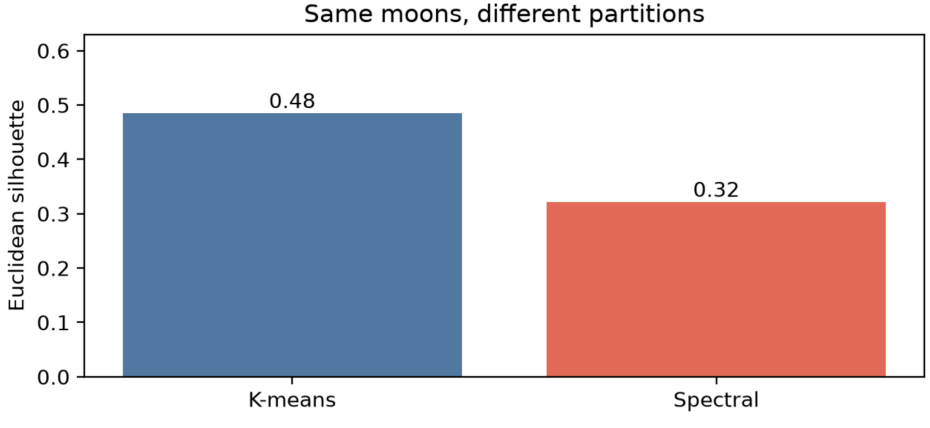
Added noise 0



## A good score depends on the geometry

Euclidean silhouette rewards compact, separated clouds. On curved moons it may prefer the **wrong shape** because its distance notion differs from the graph notion.

Treat internal metrics as evidence about their own assumptions, not as a universal verdict.



# Common validation traps

- A high score can reflect the wrong **distance or cluster notion**.
- Choosing parameters on a reference label set and then reporting ARI on those same labels overstates external evidence.
- High stability can preserve a consistently wrong answer.
- Inspect outliers, cluster sizes, and sensitivity to feature scaling and graph settings.

# A practical decision sequence

1. State what a useful group means for this dataset.
2. Inspect scaling, geometry, graph connectivity, and cluster sizes.
3. Compare plausible $k$ and graph settings with internal scores **and plots**.
4. Use ARI only when trusted labels match the goal.
5. Perturb data and parameters; report instability.

**Final question:** would these groups support a decision or explanation?